

VAHDAM Lifecycle OS

Two-mode AI operating system for CRM, retention, analytics, creative generation and knowledge-base intelligence

Budget-Friendly Output

Max Power Output

Owner emails: anchit.tandon@vahdam.com for Claude/Vahdam; anchit.tandon@gmail.com for ChatGPT, Gemini and personal tooling.

Critical principle: no app can invisibly read API keys. The correct pattern is a write-only input UI that sends the key once to a server action, stores it in Vercel environment variables, never echoes it, never logs it, then validates through a one-call health check.

1-page operating model

What must be built before any cosmetic UI work

North Star

Revenue per visitor, 6-month LTV and order #2+ contribution margin. First order may lose money; the OS must protect payback windows.

Two deployment modes

Same product logic and UI. Different environment variables, model routing, token budgets, quality gates and media limits.

Security posture

No browser-side secrets. API keys enter through a secure admin form, are encrypted in transit, stored in Vercel env vars or secret store, and masked

Data vertical merge

Fold all VAHDAM-related data engines, mailer tools, landing-page references, creative systems and analytics modules into one Data Analysis vertical.

Knowledge base

Local scan agent indexes repos and files matching VAHDAM, tea, coffee, SKU, Klaviyo, Shopify, ad, LP, creative and retention keywords into Supabase + graph layer.

Execution constraint

Claude Code/Vercel CLI should execute locally. This document is the master prompt and PRD; direct deployment from this chat is not possible.

Feature PRD - core modules

Build as verticals, not scattered pages

Admin + Key Vault

Provider setup wizard, write-only secrets, health checks, usage caps, provider status.

Mailer Studio

Brand-locked HTML email builder, star widget, Klaviyo routing, fatigue controls.

Lifecycle Calendar

Audience segmentation, dispatch windows, smart sending, holiday suppression, sequencing.

Retention Analytics

Cohorts, RPV, AOV, CM1/CM2, LTV, CAC payback, order #2 conversion.

Data Analysis Vertical

Shopify, Klaviyo, Amazon, ads, logistics, landing page and creative performance analysis.

Knowledge Base + Graph

Supabase pgvector + graph memory for SKU, segment, ingredient, asset and campaign relationships.

Creative + Media Lab

Image, video, music, voice and landing page copy generation via model cascade.

Prompt Library

Three master prompts for ChatGPT Plus, Gemini Pro and Claude Max, plus app settings instructions.

Architecture - same app, different model policy

Refactor variable names; do not fork business logic

Budget route

Default: Gemini Flash-Lite / Grok Build / local Ollama. Use caching, batch jobs, summaries and strict token ceilings.

Max-power route

Default: Claude Fable / GPT-5.5 / Gemini Pro / Grok 4.3 with Perplexity-style web research and premium media tools.

Frontend
Next.js + Vercel

API Orchestrator
Provider Router

AI Providers
OpenAI/Claude/Gemini/Grok

Data Layer
Supabase + pgvector

Graph Memory
Graphify/Neo4j layer

Model routing matrix

Use model cascade by job class, not model loyalty

Bulk classification	Gemini 3.1 Flash-Lite / Ollama	Claude Haiku/Sonnet + audit	Cheap tokens; deterministic schema output
CRM strategy	Gemini Pro or GPT mini	Claude Fable 5 + GPT-5.5 check	Best reasoning + brand judgement
Mailer HTML/code	Grok Build + Claude	Claude Fable / Sonnet + visual QA	Agentic coding and low hallucination
Search/research	Gemini grounding / xAI web	Perplexity + OpenAI web search	Fresh citations and competitor context
Images	Gemini/Flash image or low-cost API	GPT Image / Gemini Pro Image / Grok Imagine	Quality vs cost switching
Video	Static CSS/low-res Kling trial	Kling Ultra / Grok Imagine Video	Use only for paid winning ads
Music/audio	Suno free for testing only	Suno Pro/Premier	Free tier lacks commercial rights
Voice/Hey Yaara	Ollama + local TTS / xAI STT	GPT Realtime / xAI Voice + local fallback	Latency and privacy balance

Current pricing anchors

Official pricing sources where available; prices can change

OpenAI GPT-5.5		Gemini 3.1 Pro Preview	
	\$5 input / \$30 output per 1M tokens Frontier reasoning		\$2 input / \$12 output per 1M tokens Deep multimodal reasoning
OpenAI GPT-5.4 mini		Grok 4.3	
	\$0.75 input / \$4.50 output per 1M tokens Budget sensitive		\$1.25 input / \$2.50 output per 1M tokens Most general + social/web adjacent
Claude Fable 5		Grok Build 0.1	
	\$10 input / \$50 output per 1M tokens Top strategic		\$1 input / \$2 output per 1M tokens Genetic coding
Claude Sonnet 4.6		Suno	
	\$3 input / \$15 output per 1M tokens Balanced		\$0 free; Pro \$8/mo; Premier \$24/mo Music. Free plan: no commercial
Gemini 3.1 Flash-Lite		Ollama	
	\$0.25 input / \$1.50 output per 1M tokens High-volume tasks		\$0 local; Pro \$20/mo; Max \$100/mo Private/local fallback + cloud sync

Sources: OpenAI API pricing; Anthropic Claude API pricing; Google Gemini API pricing; xAI models/pricing; Suno pricing; Ollama pricing. Validate again before production spend caps.

API key integration pattern

What Claude Code must implement once, then reuse across every project

1. Provider registry

providers.json stores name, env var, required scopes, dashboard URL, test endpoint, allowed project mode.

2. Write-only input

Admin enters key in password field. UI never displays stored value; after save it shows masked last 4 chars only if provider allows.

3. Server action

POST /api/admin/secrets validates auth, writes encrypted secret via Vercel API/CLI, never console.logs body.

4. Health check

Immediately call a tiny endpoint: list models / no-op completion. Store status, latency and timestamp.

5. Mode mapping

/output-and-budget-friendly and /max-power read different env names but share the same provider adapter interface.

6. Spend guard

Monthly budget, per-run token budget, per-provider fallback order, retry limit, and kill switch.

Required env naming: BUDGET_OPENAI_API_KEY, BUDGET_GEMINI_API_KEY, BUDGET_XAI_API_KEY, BUDGET_SUNO_API_KEY, MAX_OPENAI_API_KEY, MAX_ANTHROPIC_API_KEY, MAX_GEMINI_API_KEY, MAX_XAI_API_KEY, MAX_KLING_API_KEY, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY.

Data + knowledge-base ingestion

Local agent handles files; app handles normalized memory

Local scan scope

Scan local workspace for file names and contents containing VAHDAM, vahdam, tea, coffee, SKU, Shopify, Klaviyo, Yotpo, Meta, Google Ads, Amazon, landing page, LP, PDP, email, creative, retention.

Supabase tables

documents, chunks, embeddings, campaigns, assets, products, audiences, orders, model_runs, secrets_audit, provider_health, cost_ledger.

Graph memory

Nodes: SKU, Product, Ingredient, Segment, Geography, Campaign, Asset, Mailer, Review, Claim, Source. Edges encode belongs_to, improves, caused_by, targets, references, contraindicates.

RAG rules

Every generated output must cite source chunks internally, check against VAHDAM brand rules, reject unsupported claims, and log the model path used.

Merge instruction

Remove standalone VAHDAM data-related files only after their modules, schemas, assets and prompt logic are merged into the Lifecycle OS data vertical and committed.

Quantified token + volume plan

Use audience size and campaign volume to size costs

Monthly tokens = audiences x segments x variants x channels x prompt_tokens + generations x output_tokens + QA passes x review_tokens

Audience records analysed	100,000/mo	classification summaries only
Segments	9 lifecycle cohorts	S1-S9 + suppression logic
Campaign variants	9 segments x 3 variants x 2 channels	54 outputs per campaign set
Avg input/output per mailer	3k input / 1.2k output	brand, SKU, segment and source context
Monthly generation load	54 x 4 weekly sets	216 copy outputs
Estimated generation tokens	648k input / 259k output	before QA/cache
Budget cost rough order	\$1-\$15 variable model cost	excluding paid media tools
Max-power cost rough order	\$20-\$150+ variable model cost	depends on images/video/research calls

Hard guardrail: use cached brand/system prompts, batch analysis, retrieval compression, and local Ollama for non-sensitive low-value tasks. Never rely on unlimited subscription usage for production workloads.

Three master prompt variants

Same PRD, adapted to each model strength

ChatGPT Plus prompt

Use for product reasoning, QA, implementation reviews and final synthesis. Enforce structured outputs, unit economics, security, source-aware claims and Vercel env separation.

Gemini Pro prompt

Use for large-context ingestion, multimodal data analysis, spreadsheet/log reconciliation and Google-grounded research. Demand quantified tables and source conflict checks.

Claude Max prompt

Use for Claude Code execution, repo refactor, PRD-to-code implementation, Vercel CLI setup and UI polish. Give strict file-edit boundaries and change logs.

All three prompts must include: VAHDAM brand rules, confidence tags where useful, no key exposure, no fabricated files/selectors, model fallback cascade, budget mode vs max-power mode, Supabase + graph memory, mobile analytics fallback, and change-log requirement.

Claude Code execution prompt - compressed

Paste this as the implementation brief

Build VAHDAM Lifecycle OS as a Next.js/Vercel app with two route modes: /output-and-budget-friendly and /max-power. Do not fork product logic; use provider policies and env variable maps. Implement: Admin Key Vault, provider registry, model router, Supabase + pgvector KB, graph memory adapter, lifecycle CRM modules, mailer studio, Klaviyo/Shopify/Meta/Google/Amazon connectors, data analysis vertical, creative/media lab, prompt library and mobile analytics fallback. Secrets must be write-only: never echo, log or expose API keys. Use anchit.tandon@vahdam.com for Claude/Vahdam setup notes; use anchit.tandon@gmail.com for ChatGPT/Gemini/personal provider setup notes. Scan local workspace for VAHDAM/tea/coffee/SKU/Shopify/Klaviyo/email/LP/creative/retention keywords and merge relevant data projects into the data vertical before removing standalone files. Deploy through Vercel CLI and set env vars separately for budget and max-power modes. Output a change log ending with: No other changes made.

Deployment commands

```
vercel link
vercel env add BUDGET_OPENAI_API_KEY production
vercel env add MAX_ANTHROPIC_API_KEY production
vercel --prod
# validate both routes with provider health checks
```

Done criteria

Both routes load, all providers show health status, no secrets appear in browser/source/logs, Supabase migrations pass, mobile tables render cleanly, and mailer/star widget routes validate.

Source notes and verification gates

Refresh pricing before spend-cap approval

OpenAI API pricing: GPT-5.5, GPT-5.4 mini, web search and container rates.

Anthropic Claude API pricing: Fable 5, Opus, Sonnet and Haiku rates.

Google Gemini API pricing: Flash-Lite, Pro Preview, grounding and image pricing.

xAI docs: Grok 4.3, Grok Build, Voice API and Imagine API prices.

Suno pricing: free daily credits, no commercial use on free plan, Pro/Premier rights.

Ollama pricing: local/offline availability, Pro and Max cloud usage tiers.

Verification before code merge

1. Run secret-leak scan. 2. Validate Vercel env by target route. 3. Run provider health checks. 4. Compare generated claims against KB source chunks. 5. Render mobile analytics at <768px. 6. Log token/cost ledger for every model run.